

Characterizing Multi-Host CXL 2.0: Latency, Bandwidth, and Synchronization Costs

Laurin Martins
DBIS

TU Ilmenau
Ilmenau, Germany
laurin.martins@tu-ilmenau.de

Andreas Becher
TU Ilmenau

Ilmenau, Germany
andreas.becher@tu-ilmenau.de

Alexander Baumstark
TU Ilmenau

Ilmenau, Germany
alexander.baumstark@tu-ilmenau.de

Kai-Uwe Sattler
TU Ilmenau

Ilmenau, Germany
kus@tu-ilmenau.de

Abstract

Compute Express Link (CXL) is an interconnect that extends the traditional memory hierarchy beyond a single machine. It enables disaggregated memory architecture, where memory can be pooled and shared across multiple hosts effectively. Previous systems and approaches were limited to CXL 1.0 which is restricted to intra-host scaling of memory resources with no out of the box support for sharing across multiple hosts. We evaluate a commercially available CXL 2.0 memory switching platform for multi-host shared memory. While prior work has largely focused on emulation, FPGA-based or non-available prototypes, we investigate the real-world performance and system implications of deploying CXL as shared memory across multiple hosts on a commercially available setup. Our evaluation reveals fundamental trade-offs: Access to shared CXL memory exhibits up to 4x higher latency and significantly lower bandwidth compared to local DRAM, but enables flexible memory pooling with fair sharing across hosts. More importantly, we show that the lack of hardware cache coherence in CXL 2.0 introduces non-trivial challenges, requiring carefully chosen instruction patterns.

Keywords

CXL, Shared Memory

ACM Reference Format:

Laurin Martins, Alexander Baumstark, Andreas Becher, and Kai-Uwe Sattler. 2026. Characterizing Multi-Host CXL 2.0: Latency, Bandwidth, and Synchronization Costs. In *22nd International Workshop on Data Management on New Hardware (DaMoN '26)*, May 31–June 05, 2026, Bengaluru, India. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3789237.3809131>

1 Introduction

Memory disaggregation promises to improve resource utilization in data centers by pooling memory across servers, allowing compute and storage to scale independently. Several packet-oriented interconnect technologies enable a form of memory disaggregation:

RDMA over Converged Ethernet (RoCE) provides low-latency remote memory access but requires specialized networking hardware; NVMe-oF allows memory devices to be accessed over standard Ethernet but introduces protocol overhead; and PCIe-attached memory expanders increase local capacity but remain tied to a single host.

A recent development is Compute Express Link (CXL), an open standard built on the PCIe physical layer that defines protocols for memory-semantic access to remote devices. CXL 1.0 and 1.1 introduced the foundational protocols but restricted topologies to single-host configurations. CXL 2.0 added switching capabilities, enabling multiple hosts to access shared memory pools through a CXL switch. Figure 1 shows a typical CXL 2.0 topology with two hosts connected to pooled Type 3 memory devices via a switch. However, CXL 2.0 Type 3 devices provide no cache coherence across hosts. Unlike traditional multi-socket NUMA systems where cache coherence protocols (e.g., MESI, MESIF) ensure writes are automatically visible across processors, CXL 2.0 requires explicit software management. When one host writes to shared CXL memory, that write may remain in the host's CPU cache indefinitely unless explicitly flushed using instructions such as CLFLUSH. This fundamental difference raises questions about the practical costs of synchronization in CXL-based shared memory systems. Existing frameworks explicitly targeting CXL such as FAMFS [6] and SMDK [7] abstract this non-coherency by presenting CXL memory as a storage tier, filesystem, or NUMA node, minimizing application changes. While these abstraction layers are valuable for compatibility, they obscure the underlying hardware characteristics. Understanding the raw synchronization costs, such as which instructions provide visibility, the amount of overhead that cache management introduces, and whether cache-bypassing helps, is essential for optimizing existing frameworks and designing new CXL shared memory systems.

In this paper, we characterize the baseline performance characteristics and synchronization mechanisms of multi-host CXL 2.0 memory access using commercially available hardware (AMD EPYC 9455P, Falcon C5022 switch, Micron CZ122 memory expander). Initially, fundamental memory metrics are measured, including single-host latency, bandwidth, and loaded latency, in order to establish a baseline performance. Subsequently, we assess multi-host contention scenarios in which both hosts access CXL memory concurrently without coordination, quantifying the impact on latency and bandwidth. Finally, the write visibility latency for coordinated



This work is licensed under a Creative Commons Attribution 4.0 International License. *DaMoN '26, Bengaluru, India*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2455-8/26/05

<https://doi.org/10.1145/3789237.3809131>

access across different x86 cache management instructions, memory barriers, and store operations is systematically measured. A comparison is made between cache-coherent stores with explicit flush against cache-bypassing instructions, non-temporal, and vector stores. The contributions of our paper are as follows:

- (1) We evaluate a commercially available CXL switch enabling true memory pooling and multi-host shared memory via CXL 2.0, providing insights unavailable from FPGA-based or single-host studies.
- (2) We systematically characterize suitable synchronization operations, showing that explicit cache invalidation is mandatory for reads, that multiple store variants (temporal, non-temporal, atomic) achieve similar performance when explicitly flushed, while direct stores exhibit significantly higher latency.

2 Background & Related Work

Hardware and systems like InfiniBand with RDMA and CXL have made memory disaggregation a key research focus, aiming to separate computation from memory to surpass traditional limits. However, RDMA-based systems suffer from high latency through RDMA round trip times and protocol overhead ($> 100\mu s$) [5, 12].

2.1 Compute eXpress Link

CXL is an interconnect standard for CPUs, memory, and accelerators built on PCIe, offering new opportunities to improve scalability. CXL uses the same physical links as PCIe but runs a different protocol and needs explicit support from the memory controller. CXL 1.0 defines three protocols: CXL.io (device setup and I/O), CXL.cache (cache coherency), and CXL.mem (memory access) with three device types: Type 1–2 (accelerators without/with memory), and Type 3 (memory expanders). Type 3 memory expander devices can be configured in a host as an additional NUMA (non-uniform memory access) node or as a DAX (Direct Access) device. CXL Type 3 devices based on CXL 1.0/1.1 have already been evaluated for single-host memory expansion [10, 14], demonstrating substantially higher access latency and the need for efficient data placement and interleaving techniques to mitigate its impact. CXL 2.0 introduces *CXL switches* to scale CXL devices within single-level hierarchies, thereby enabling device scaling, memory pooling, and fabrics. Emerging memory appliances with CXL 2.0 switching capabilities enable scalable memory pooling and sharing, such as the H3P Falcon C5022 [2]. CXL 3.0 introduces multi-level switching, memory pooling, and cache-coherent memory sharing, and is based on PCIe 6.0, which is not yet widely available at the time of writing. Previous work evaluated emulated CXL shared memory for distributed join operations in DBMS [1].

In this work, we therefore focus on real CXL 2.0 and Type 3 devices. The first evaluation of prototypical (FPGA-based) [13, 14], commercially unavailable appliances [3, 11], or appliances with closed feature sets [4] shows superior performance compared to traditional RDMA-based baselines. However, none of these works show the impact of non-cache-coherent CXL 2.0 memory sharing on data access of commercially available appliances. At the time of writing this paper, the software stack for CXL shared memory is limited by the available features of recent Linux kernels. Device

type 3 libraries, such as SMDK or HMSDK [7, 9], are not sufficient, as they are both designed to extend and tier memory, not to provide true, transparent memory sharing. FamFS offers a filesystem to manage and use shared memory across CXL-connected hosts [6]. However, it introduces additional overhead for file management. Therefore, we focus in this paper on the practical implications when using memory sharing on instruction-level and cache behavior.

2.2 Memory Access

Memory access latency and bandwidth are primarily determined by the position of data within the memory hierarchy and the mechanisms used to retrieve them. Modern micro-architectures rely on a three tier cache hierarchy to hide the high latency of main memory by exploiting spatial and temporal locality. Multi core and multi socket systems introduce another source for latency differences. Due their positioning some components are physically farther apart incurring additional latency. Although it enables byte-addressable memory access like DRAM, CXL 2.0 operates outside the established cache-coherency domains, leading to higher latency due to longer data paths and protocol overhead. Further, it requires explicit mechanisms, such as cache line flushes or non-caching access, to ensure visibility of updates to shared memory when such changes need to be visible as soon as possible.

Modern x86 comes with numerous store operations that trigger different caching behavior. To load data from CXL memory, a regular MOV operation can be used. But due to the aforementioned missing cache-coherency by design, it is certain that changes to the actual value are not visible to other hosts as long as the address is not evicted from the local cache. Therefore, we evaluate the following classes of store (MOV) operations in this work. The *regular* store, store using a *non-temporal* hint, the *direct* store, *vector* stores, and XCHG *atomic* store. Additionally, we assess the impact of combining them with different flush (CLWB, CLFLUSH, CLFLUSHOPT) and fence (SFENCE, LFENCE) operations. In this work, we investigate baseline performance characteristics and the impact of x86 store and load instruction variations on access latency to CXL 2.0 shared memory, provided via a CXL switch, in single host (pooling) and multi host setups.

3 Hardware Setup & Software Configuration

Figure 1 shows the hardware setup in our lab used for all evaluations conducted for this work. Two similar hosts (CXL-a, CXL-b) are connected via CDFP cables to the CXL host (Falcon C5022) which operates as a CXL switch and serves CXL shared memory to the connected hosts. The detailed specification of the hardware setup is listed in Table 1.

CXL Switch. The CXL switch is shipped as an appliance and provides one of the first CXL 2.0 memory pooling and sharing capability. Internally, the appliance runs on an energy efficient CPU with its own DRAM and a common server OS. It is capable to serve up to 22 CXL memory expansion cards in the E3.S form factor, providing several TBs of memory to connected hosts. Hosts are connected through PCIe via CDFP cables and retimer cards, where the current configuration of the C5022 is able to connect up to 4 hosts. However, the switch supports the connection of further switches in a cascade. The switching functionality is enabled by the

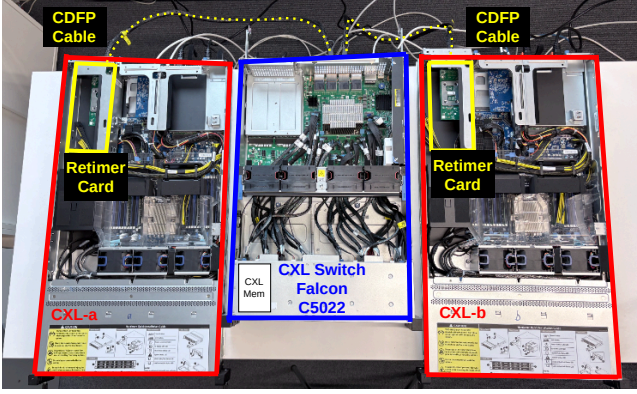


Figure 1: Physical hardware setup of the switch and the connected hosts.

Component	CXL-a/b	CXL Switch
Model	Gigabyte R263	Falcon C5022
CPU	AMD EPYC 9455P	Intel Atom C3558
Cores	48	4
Cache	L1d: 2.3 MiB L2: 48 MiB L3: 256 MiB (shared)	–
RAM	4×16 GB DDR5 @ 6400 MT/s	2×8GB DDR4
CXL Switch	–	Xconn XC50256 B2
CXL Memory	–	1× Micron CZ122 (256GB)
Extra	PCIe5×16 Retimer card (CDFP)	PCIe5×16 CDFP
OS	Ubuntu 24.04 (kernel 6.8.0-100-generic)	CentOS 7 (kernel 6.6.4)

Table 1: Configurations of evaluation platforms.

XConn C50256 B2, a high-performance CXL 2.0 switch ASIC that provides dynamic memory pooling and sharing across connected hosts. It implements a fully non-blocking fabric, allowing multiple hosts to access pooled memory resources concurrently. Our setup includes a single Micron CZ122 CMM-D memory expansion card with 256 GB capacity, directly connected to the C5022 appliance.

For management, the appliance provides a web GUI as well as CLI tools (*xconn-control*) to assign CXL memory devices to connected servers. It is possible to assign individual CXL devices to multiple servers at the same time, which allows for memory sharing, albeit without any form of automatic cache coherence. At the time of writing this paper, all changes to the CXL topology must be made before any connected host is powered on. Changes during runtime are currently not supported.

Hosts. Our setup consists of two Gigabyte servers equipped with an identical hardware setup, as listed in Table 1. For both servers, the BIOS and OS must be configured to support the CXL protocol and need to be compatible with the requirements of the C5022

appliance. Each server is connected via PCIe to a CDFP retimer card, which establishes the physical and protocol-level link to the CXL switch through a CDFP cable operating under the CXL 2.0 standard. The retimer card plays a crucial role in maintaining signal integrity over the link by compensating for signal degradation and enabling stable communication across longer cable distances. This allows the servers to interface with the external CXL switch as if it were part of the local PCIe hierarchy. Through the CXL protocol stack, it enables hosts to access pooled memory resources attached to the switch.

Additional OS configuration steps are required to make the CXL-attached memory usable by user space programs, such as enabling appropriate kernel modules and configuring the memory mode.

Using NUMA-mode, exposes the CXL memory as an additional CPU-less NUMA node that is managed by the operating system.

We use DAX-mode rather than NUMA-mode to isolate raw hardware behavior. Using NUMA-mode introduces kernel overhead (e.g., page faults, TLB shootdowns, NUMA balancing) that obscures the CXL link characteristics we aim to measure. DAX provides direct byte-addressable access while bypassing the page cache, allowing us to characterize CXL performance without OS interference.

In our setup, we reconfigure the memory device as *devdax* via the *daxctl* utility, which exposes it as a character device, e.g., as */dev/dax0.0*. This allows user space applications to map the memory directly into their address space using the *mmap* syscall, bypassing traditional kernel memory management.

4 Baseline Characterization

This section investigates the performance implications of using CXL-shared memory via a CXL switch, as described in Section 3. A single CMM-D device is connected to the CXL switch and assigned to both connected hosts as a shared resource. We evaluate latency, bandwidth, and switching implications, and contrast these results with traditional local DRAM to highlight trade-offs.

4.1 Single-Host Performance

We evaluate a single host accessing CXL shared memory, served by the CXL host, to evaluate the performance of the link between a single host and the switch. We use a ¹fork of Google’s ²multichase benchmark to assess the latency, bandwidth, and latency under load of each host to the CXL shared memory. The benchmark can be configured to set a file descriptor as the backing memory for its working set and benchmark target. This makes it easy to switch between RAM and CXL backing memory during test execution. We measure the access characteristics from both hosts of our setup (CXL-a, CXL-b), but they are not accessing the memory at the same time.

Accessing CXL shared memory. When the OS has configured the CXL shared memory as a DAX device the software needs to open a file descriptor and *mmap* that file descriptor into its address space.

Listing 1: Mapping a DAX device using *mmap*

```
int fd = open("/dev/dax0.0", O_RDWR);
void *addr = mmap(NULL, size,
```

¹<https://github.com/LaurInMartins/google-multichase>

²<https://github.com/google/multichase>

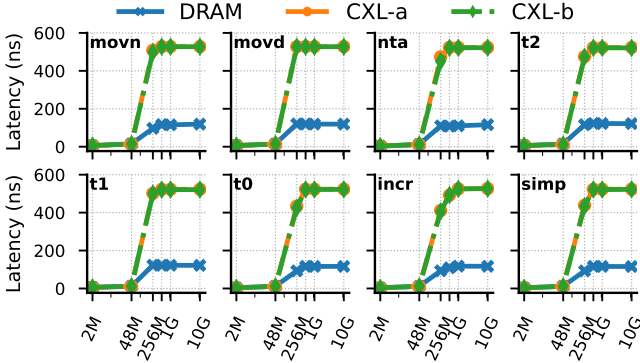
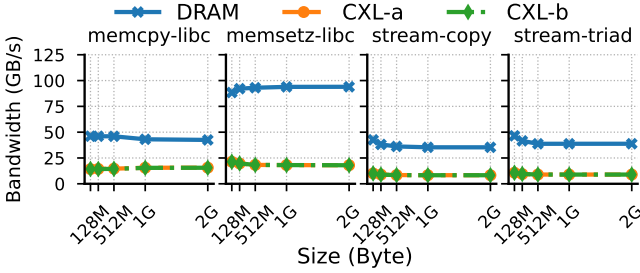
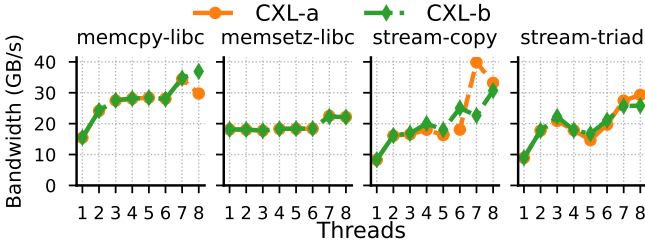


Figure 2: Intra-host latency evaluation via pointer-chasing methods.



(a) Single-threaded bandwidth across varying data sizes.



(b) Multi-threaded bandwidth with varying number of threads.

Figure 3: Intra-host bandwidth evaluation.

```
PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
```

Listing 1 shows the code used throughout this paper to map the DAX device into user address space. We deliberately avoid flags like `O_SYNC` and `MAP_SYNC` to isolate the raw hardware behavior without OS optimizations. It is important to note that the `MAP_SHARED` flag is required for multi-host visibility, thus must be present.

4.1.1 Latency. The *multichase* latency test generates a chase load pattern that is then used for pointer chasing traversal (random or ordered) through the generated data.

The *simp* pattern exposes raw memory latency via dependent pointer chasing, while *incr* uses sequential accesses that benefit from hardware prefetching. The *t0-t2* variants apply software prefetching at different cache levels, and *nta* uses non-temporal accesses to reduce cache pollution. The *movd* and *movn* patterns

represent SIMD-based loads with and without cache utilization. The benchmark is a measurement of the number of jumps that have occurred within a specified time interval. It calculates the average latency of the traversals per chase thread. We run the benchmark only with 1-8 threads, to restrict caching effects. Figure 2 shows that, using a load size of less than the size of the CPUs L3 cache size (cf. Table 1) traverses through the cache for most of the test’s runtime. With sizes above the L3 cache size, CXL exhibits a consistent latency overhead compared to DRAM, largely independent of the working set size once it exceeds cache capacity, reaching up to approximately 550 ns, compared to 120 ns of DRAM.

Result: Access to CXL shared memory exhibits $\sim 4\times$ higher latency than DRAM.

4.1.2 Bandwidth. For the bandwidth evaluation in Figure 3, we use *multiloop* with different parallel workloads. The *multiloop* test creates a load per thread that will be used for various combinations of load and store operations. Each thread reports the amount of data transferred. The *memcpy* workload performs bulk data copying, *memset* initializes a memory region to a fixed value, *stream-copy* is a simple vector kernel that copies one array to another in parallel, and *stream-triad* performs a fused operation of $A = B \times scalar \times C$ in parallel.

Figure 3a shows the single threaded bandwidth using varying data sizes, located in CXL shared memory, compared to DRAM bandwidth. For all workloads, the maximum bandwidth of CXL memory, served by the CXL switch, lies in the range of 20-25 GB/s.

In Figure 3b, we evaluate multi-threaded bandwidth for each host. Here, the maximum achievable bandwidth is 31 GB/s, which is the theoretical limit of the CXL memory device, which is around a fourth of DRAM.

Result: Access to CXL shared memory reaches only 25% of DRAM Bandwidth

4.1.3 Latency under Load. In the benchmark of Figure 4, the interaction between memory bandwidth workloads and concurrent pointer chasing is evaluated. It creates latency and bandwidth threads and runs them simultaneously from the same host. We use the same workloads as for the bandwidth evaluation. Under concurrent memory load, CXL exhibits increased access latency when increasing the number of threads which increases the memory load. Latencies range from approximately 760–770 ns for *memcpy* up to 1.3 μ s for *memset*, indicating that write-heavy workloads introduce the strongest interference. Streaming patterns such as *stream-copy* (970 ns) and *stream-triad* (930 ns) fall in between, while *memcpy* remains the least affected.

Result: CXL shared memory exhibits up to $2\times$ higher latency under high load.

4.2 Multi-Host Contention

The objective of these benchmarks is to quantify the impact of concurrent access to the CXL shared memory from multiple hosts. We differentiate between *non-cooperative* and *cooperative* workloads. In the context of non-cooperative operations, both machines aim to achieve either the lowest latency or the highest bandwidth while sharing the resource with the other machine, but not accessing the same memory address in CXL concurrently. Cooperative means

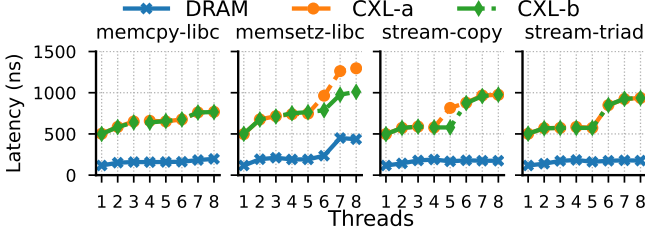


Figure 4: Intra-host loaded latency evaluation

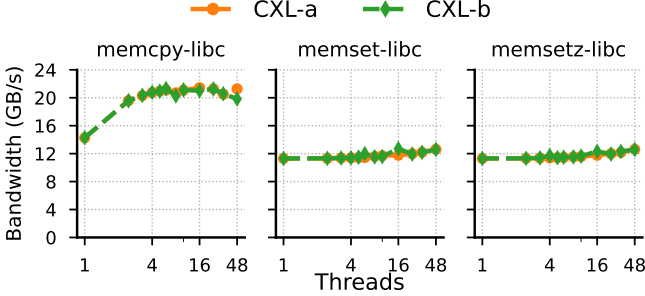


Figure 5: Inter-host non-cooperative bandwidth evaluation using multiple threads

that the CXL shared memory is utilized for the purpose of data exchange between the two hosts without the use of other out-of-band communication.

The *multiload* benchmark includes an offset flag that allows moving the entire workload by the specified offset. This feature allows us to run the same benchmark from different hosts, ensuring that they do not disrupt the benchmark data of the other host.

Both machines run the same bandwidth benchmark as described in Section 4.1.2. We only show the *memcpy*, *memset* and *memsetz* workloads, as the other workloads are dominated by local cache effects and do not provide further insights into CXL shared memory performance.

Figure 5 show the results for the non-cooperative bandwidth benchmark. The results show that for the *memset* and *memsetz* variants the bandwidth is evenly split between hosts. For the *memcpy* workload, the bandwidth is similar to the single host results indicating almost no impact for streaming access patterns. Overall, the link saturation is almost reached with a single thread running on both hosts. However, further tests are required with more than two hosts. In the tested setup, the switch exhibited fair arbitration, with each host achieving evenly split bandwidth, peaking at ~12GB/s.

Result: Available CXL bandwidth is evenly distributed across competing hosts.

In the following, we evaluate cooperative works via cross-host synchronization, where multiple hosts operate on the same memory address located in CXL shared memory.

Listing 2: Pseudocode of Flag Exchange Test

```

loop Host A {
  CxlWrite(flag, 1)
  CxlAwaitExternalChange(flag, 0)
}

```

```

}
loop Host B {
  CxlWrite(flag, 0)
  CxlAwaitExternalChange(flag, 1)
}

```

5 Cross-Host Synchronization

Due to CXL 2.0’s lack of hardware cache coherency, explicit software management for cross-host coordination is required. To support this, we systematically evaluate which combinations of x86 store and load operations provide reliable write visibility and evaluate their latency.

5.1 Flag Exchange Test

We implement a symmetric flag exchange benchmark where two hosts continuously toggle a shared flag stored in CXL shared memory between 0 and 1. Listing 2 shows the flag exchange test in pseudocode. `cxlMem` denotes the address of the shared flag in CXL memory, and `v` represents the expected flag value. The operation `CxlWrite(cxlMem, v)` stores the value `v` at address `cxlMem` using a combination of a selected MOV variant, a FLUSH variant, and a FENCE variant. The operation `CxlAwaitExternalChange(cxlMem, v)` implements a busy-wait loop that repeatedly loads from `cxlMem` until the observed value equals `v`. If the value does not match, a FLUSH variant is issued before retrying.

Each iteration requires both hosts to propagate a write to CXL memory and to invalidate any stale cached copy in order to observe the peer’s update. To study the impact of different instruction choices, we evaluate combinations of move, flush, and fence operations. For stores on CXL memory, we consider regular instructions (MOV), non-temporal variants (MOVNTI, VMOVNTDQ), direct stores (MOVDIRI, MOVDIR64B), and atomic exchanges (XCHG). For cache management, we evaluate no explicit flushing as well as CLWB (cache line write-back), CLFLUSH, and CLFLUSHOPT, applied on both the write and read paths. Finally, to evaluate the impact of explicit ordering, we compare runs without any explicit fence instruction to those using SFENCE for store and LFENCE for load ordering.

To prevent undesired compiler optimizations, we declare the shared flag as volatile. This ensures that each load and store operation is emitted as specified in the benchmark logic and is not eliminated, cached in registers, or reordered by the compiler. In particular, the busy-wait loop in `CxlAwaitExternalChange` relies on repeatedly observing updates from the remote host, which would be invalidated if the compiler hoisted loads out of the loop or reused stale values. We note that volatile only enforces compiler-level ordering and visibility, but does not provide hardware-level memory ordering guarantees; therefore, we explicitly combine it with appropriate fence and flush instructions to enforce correct cross-host communication semantics.

After a 1,000-iteration warmup, we measure 1 million flag exchanges for each working combination, recording: (1) average round-trip latency and (2) the number of cache invalidations (spins) required per exchange. Each configuration is executed 5 times; we

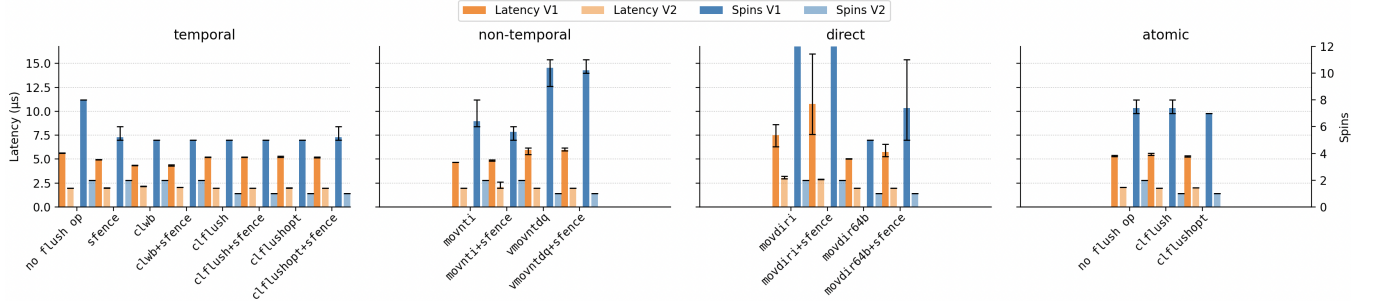


Figure 6: Inter-host cooperative latency evaluation

report the average latency in Figure 6. All code, test scripts, and raw measurement data are available in our GitHub repository³.

5.2 Results

The volatile keyword alone is insufficient to ensure correct behavior. In configurations without explicit cache invalidation, marking the flag as volatile still fails, as generated load instructions may be satisfied from the local cache if the cache line is present.

On non-coherent CXL memory, no memory ordering primitive alone prevents this behavior, making explicit cache management necessary. Figure 6 shows synchronization latencies across our tested variants and combinations. V1 shows results for the load side using CLFLUSH, while V2 shows the load side using CLFLUSH followed by an LFENCE. Exchanging CLFLUSH with CLFLUSHOPT on the load side has not exhibited significant differences in our testing.

5.2.1 Load-Side Comparison. When retrieving external changes using CLWB alone fails for *CxlAwaitExternalChange*. It writes dirty cache lines back to memory while potentially retaining the line in the cache. However, when the await loop reads the flag, the value enters the cache as a clean line. Subsequent CLWB operations have no effect on clean lines as they do not force invalidation to refetch the data from CXL shared memory.

Only CLFLUSH and CLFLUSHOPT provide reliable cross-host visibility as invalidating operations force a refetch from CXL shared memory. Both achieve nearly identical latency across all tests. At CXL latency scales, the CPU-side difference between serializing and optimized flush is negligible. Adding the LFENCE instruction between flush and load instructions yields the most desirable results, as this variant reduces spins to one, and consistently leads to latency to $\sim 2\mu s$ time for a flag exchange for most variations except direct stores. The addition of this fence instruction on the load side ensures that speculative execution does not occur, mitigating busy-wait behavior.

Result: Explicit cache invalidation (CLFLUSH(OPT)) is required for correct cross-host visibility on CXL memory. Combining them with an explicit LFENCE instruction yields the lowest latency results.

5.2.2 Store-Side Comparison. When using no fence on the load-side, temporal stores with explicit cache invalidation achieve the best performance, reaching approximately $\sim 5\mu s$ latency with ~ 6

spins per iteration. The use of SFENCE has no measurable impact on latency (remaining around $\sim 5\mu s$), as ordering is already enforced by the cache flush operations. Once the LFENCE is added to the load-side latency drops to the same value across all temporal variations, using CLFLUSH(OPT) drops the spins to 1 making those variations the most efficient. The non-temporal store bypasses cache and writes directly to memory, avoiding write-back overhead entirely.

Looking at the load-side without the LFENCE instruction (V1), using MOVNTI performed better than average in this test. The vector variant incurs more spins until a change becomes visible even though latency remains on par with other variants.

They exhibit significantly more cache invalidations per exchange (15-25 spins vs 6 for temporal stores). This indicates slower write propagation to CXL memory. The high variance in both latency and spin count even between CLFLUSH and CLFLUSHOPT used within the loop, suggests direct store paths are less predictable for CXL synchronization.

For the groups of stores, the SFENCE variants exhibit no significant impact on either latency or spin count. Adding a LFENCE on the load-side (V2) drops spins and latency to results on par with temporal store variants. In our tests the V2 results for vmovntdq exhibit the same latency and spins as the temporal stores.

In contrast, direct stores significantly increase visibility delay, requiring up to ~ 25 spins, indicating slower propagation and higher variability. Using LFENCE on the load side transforms the results significantly. With it in place the movdi r64b variant is on par with the other fast store variants in terms of latency and spins.

XCHG performs comparably to temporal stores with similar latency but higher spin counts. In contrast to their name, direct stores show poor visibility characteristics and higher variability. Comparing variants with and without SFENCE shows negligible latency differences. The cache flush operation itself ensures ordering—writes reach CXL memory only after the flush completes. As with most store variants using LFENCE on the load-side (V2) and CLFLUSH(OPT) on the store side drops latency to $\sim 2\mu s$ and 1 spin.

These findings are specific to the flag exchange pattern on the tested hardware and software configuration. Generalization to locks, queues, and other synchronization primitives requires further evaluation.

Result: Stores with explicit invalidation provide the best latency-visibility trade-off.

³<https://codeberg.org/Laur1n/damon-2026-cxl>

5.3 CXL Mutual Exclusion

Based on the results presented in Section 5.2 we derive load and store primitives specifically for CXL shared memory. Using them guarantees cache bypassing behavior and the fastest way to observe changes to shared memory by another host. This is especially useful for distributed mutual exclusion when access to shared resources needs to be coordinated to ensure that access to critical sections happens sequentially.

Peterson’s mutual exclusion algorithm [8] is a prime candidate for CXL shared memory as it does not rely on the existence of atomics. We implement it using the CXL primitives shown in Listing 3. Listing 4 shows the pseudo-code of the implementation.

Listing 3: Pseudocode of CXL primitives

```
cxl_write (addr, v) {
    *addr = v
    CLFLUSHOPT(addr)
    SFENCE
}
cxl_read (addr) -> v {
    CLFLUSHOPT(addr)
    LFENCE
    return *addr
}
```

Listing 4: Pseudocode of Peterson’s Algorithm using CXL primitives

```
init {
    bool flag[2] = {0,0}, turn = 0
}
lock (id) {
    cxl_write(flag[id], true)
    cxl_write(turn, !id)
    while (cxl_read(flag[id])
        && cxl_read(turn) == !id) {}
}
unlock (id) {
    cxl_write(flag[id], false)
}
```

We measured the latency for the lock implementations. Similar to the previous experiment we run 1 Million iterations, measure the overall runtime, and calculate the average runtime per iteration. In the critical section, the conducted test loads a variable from CXL shared memory and stores its increment back into the CXL shared memory. This is repeated until a predetermined number is reached, in the benchmark case 1 million. Our tests showed that a single loop execution requires around $\sim 3.3\mu\text{s}$ for each host.

6 Conclusion

We characterized multi-host CXL 2.0 memory access using production hardware, establishing baseline performance and synchronization costs. The tested CXL 2.0 memory exhibits $4\times$ higher latency ($\sim 550\text{ ns}$ vs $\sim 120\text{ ns}$) and $1/4$ the bandwidth ($\sim 25\text{ GB/s}$ vs $\sim 100\text{ GB/s}$) compared to local DRAM, but it enables fast and fair memory and bandwidth sharing across competing hosts. For cross-host synchronization, temporal stores, non-temporal stores (VMOVNTDQ),

direct stores (MOVDIR64B) and the XCHG atomic store achieve comparable latency ($\sim 2\mu\text{s}$) when paired with an LFENCE on the load-side. Some store variants (MOV(CLWB), MOVNTI, MOVDIRI) exhibit higher than 1 spin counts without reducing latency, indicating inefficiency. Based on these results we implemented Peterson’s Algorithm for CXL shared memory and observed $\sim 3.3\mu\text{s}$ per iteration of the lock, critical section, unlock loop. All findings are specific to the tests on the tested hardware.

Future work should extend this characterization to higher-level primitives (multi-host locks, queues, barriers) and evaluate more workloads on CXL 2.0 shared memory. Moreover, our evaluation is subject to several limitations. First, experiments were conducted with only two connected hosts. Incorporating additional hosts would enable a more comprehensive analysis of contention and its impact on the CXL link under realistic multi-node scenarios. Second, the system includes only a single CXL memory device. Evaluating multiple CXL memory devices would provide deeper insight into device-level parallelism, resource sharing, and their combined effect on overall scalability. Finally, we focus exclusively on DAX-based access. Evaluating NUMA-based CXL memory would allow us to study coherence behavior, latency characteristics, and system-level integration from a different perspective.

Acknowledgments

Funded by DFG SPP2377 grants BE 7385/1-2 and SA 782/31-2.

References

- [1] Alexander Baumstark, Marcus Paradies, Kai-Uwe Sattler, Steffen Kläbe, and Stephan Baumann. 2024. So Far and yet so Near - Accelerating Distributed Joins with CXL. In *Proceedings of the 20th International Workshop on Data Management on New Hardware* (Santiago, AA, Chile) (DaMoN '24). Association for Computing Machinery, New York, NY, USA, Article 7, 9 pages. doi:10.1145/3662010.3663449
- [2] H3 Platform Inc. 2025. *Falcon C5022: Composable CXL Memory Appliance*. H3 Platform. <https://www.h3platform.com/product-detail/overview/35> Accessed: 2025-02-17.
- [3] Tao Huang, Yudi Qiu, Yuxin Tang, Qixiao Liu, Wenpu Hu, Xinjun Yang, Yingqiang Zhang, Hao Chen, Guoqiang Xu, Zipeng Ouyang, Yuemin Wu, Jian Chen, Wei Wang, and Yang Kong. 2026. CXLock: Efficient and Scalable Lock Management for CXL-enabled Distributed Systems. *IEEE Trans. Comput.* (2026), 1–12. doi:10.1109/TC.2026.3658040
- [4] Han Deok Lee, Jehoon Park, Younghyun Lee, Junhyeok Im, Jin Jung, Jinin So, Siamak Tavallaee, Taehoon Lee, Woo Taek Shim, Chin-Hua Chang, Jianping Jiang, Sungwook Ryu, Taeksang Song, Wonhwa Shin, and Sangjoon Hwang. 2026. Pangaea v2: CXL-based Disaggregated Memory System Architecture for Cloud-Native Orchestration. *IEEE Trans. Comput.* (2026), 1–14. doi:10.1109/TC.2026.3654926
- [5] Teng Ma, Zheng Liu, Chengkun Wei, Jialiang Huang, Youwei Zhuo, Haoyu Li, Ning Zhang, Yijin Guan, Dimin Niu, Mingxing Zhang, and Tao Ma. 2024. HydraRPC: RPC in the CXL Era. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 387–395. <https://www.usenix.org/conference/atc24/presentation/ma>
- [6] Micron Technology. 2025. FamFS: A Scale-Out Shared-Memory File System for Fabric-Attached Memory. <https://github.com/cxl-micron-reskit/famfs>. Accessed: 2026-02-20.
- [7] OpenMPDK. 2024. Scalable Memory Development Kit (SMDK). <https://github.com/OpenMPDK/SMDK>. Accessed: 2026-02-20.
- [8] G.L. Peterson. 1981. Myths about the mutual exclusion problem. *Inform. Process. Lett.* 12, 3 (1981), 115–116. doi:10.1016/0020-0190(81)90106-X
- [9] SK hynix. 2025. Heterogeneous Memory Software Development Kit (HMSDK). <https://github.com/skhynix/hmsdk>. Accessed: 2026-02-20.
- [10] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxian Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, et al. 2023. Demystifying cxl memory with genuine cxl-ready systems and devices. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 105–121.
- [11] Xi Wang, Jie Liu, Jianbo Wu, Shuangyan Yang, Jie Ren, Bhanu Shankar, and Dong Li. 2025. Performance Characterization of CXL Memory and Its Use Cases. In

- 2025 *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 1048–1061. doi:10.1109/IPDPS64566.2025.00097
- [12] Zhonghua Wang, Yixing Guo, Kai Lu, Jiguang Wan, Daohui Wang, Ting Yao, and Huatao Wu. 2024. Rcmp: Reconstructing RDMA-Based Memory Disaggregation via CXL. *ACM Trans. Archit. Code Optim.* 21, 1, Article 15 (Jan. 2024), 26 pages. doi:10.1145/3634916
- [13] Marcel Weisgut, Daniel Ritter, Florian Schmeller, Pinar Tözün, and Tilmann Rabl. 2025. CXL-Bench: Benchmarking Shared CXL Memory Access. *Proceedings of the VLDB Endowment*. ISSN 2150 (2025), 8097.
- [14] Marcel Weisgut, Daniel Ritter, Pinar Tözün, Lawrence Benson, and Tilmann Rabl. 2025. CXL Memory Performance for In-Memory Data Processing. *Proceedings of the VLDB Endowment* 18, 9 (2025), 3119–3133.